Claude Code (Windsurf) - Complete Mode Guide

Comprehensive guide to all modes in Claude Code / Windsurf IDE and when to use them.

Mode Overview

Claude Code (Windsurf IDE) has 4 primary modes:

1. **Vibe Mode** - Conversational Q&A and guidance
2. **Plan Mode** - Planning before execution
3. **Auto Mode** - Autonomous execution
4. **Cascade Mode** - Multi-file autonomous implementation

---

1. Vibe Mode (Chat & Guidance)

What It Is

- Conversational mode for questions and guidance
- No file edits or command execution
- Read-only assistance

When to Use

- Learning new concepts
- Understanding existing code
- Design discussions
- Getting explanations
- Asking "how to" questions
- Code reviews and feedback

Capabilities

☑ Answer questions ☑ Explain code ☑ Suggest approaches ☑ Provide examples ☑ Discuss architecture
❌ No file editing ❌ No command execution ❌ No autonomous action

Examples

Example 1: Understanding Code

```
You: "Explain how this authentication middleware works"

Claude (Vibe):
"This middleware implements JWT authentication:

1. Extracts token from Authorization header
2. Verifies token signature using secret key
3. Decodes user info from payload
```

```
4. Attaches user to request object
5. Passes to next handler or returns 401

The flow is:
Request → Extract Token → Verify → Decode → Next Handler

Security notes:
- Uses HS256 algorithm
- Token expires after 24 hours
- Includes user_id and role in payload"
```

Example 2: Design Discussion

```
You: "Should I use REST or GraphQL for this API?"

Claude (Vibe):
"For your use case (mobile app with varying data needs),
here's the comparison:

REST:
+ Simpler to implement
+ Better caching
+ Familiar to most developers
- Multiple requests for nested data
- Over-fetching or under-fetching

GraphQL:
+ Single request for complex data
+ Flexible client queries
+ Strong typing
- More complex setup
- Harder to cache
- Steeper learning curve

Recommendation: Start with REST. Your data model is simple,
and you can add GraphQL later if over-fetching becomes an issue.

Would you like me to help design the REST endpoints?"
```

Example 3: Getting Help

```
You: "How do I implement rate limiting in FastAPI?"

Claude (Vibe):
"Here's how to implement rate limiting in FastAPI:

Option 1: slowapi library (recommended)
[provides example code]
```

```
Option 2: Custom middleware
[provides example code]

Option 3: Redis-based (for distributed systems)
[provides example code]

For your single-server setup, I recommend slowapi.
Would you like me to implement it? (Switch to Auto/Cascade mode)"
```

Best For

- Planning phase
- Learning
- Understanding
- Design reviews
- Non-coding discussions

---

2. Plan Mode (Plan-then-Execute)

What It Is

- Creates detailed execution plan first
- Shows plan for your approval
- Executes plan after confirmation
- Step-by-step visibility

When to Use

- Want to see plan before execution
- Complex features requiring multiple steps
- Need to verify approach first
- Learning mode (see how AI thinks)
- Critical changes (review before action)

Capabilities

☑ Create execution plan ☑ Show steps before execution ☑ Edit files after approval ☑ Run commands after approval ☑ Modify plan if needed ❌ Not fully autonomous ❌ Requires approval to proceed

Workflow

```
1. You describe task
2. Claude creates plan
3. Claude shows plan to you
4. You review and approve (or request changes)
5. Claude executes plan step by step
6. Reports results
```

Examples

Example 1: Feature Implementation

```
You: "Add user authentication to this app"

Claude (Plan Mode):
"I'll implement user authentication. Here's my plan:

📋 PLAN:

1. Database Layer
   - Create users table migration
   - Add password_hash and email fields
   - Create User model

2. Authentication Service
   - Implement password hashing (bcrypt)
   - Create JWT token generation
   - Add token verification

3. API Endpoints
   - POST /register - user registration
   - POST /login - user login
   - GET /me - get current user (requires auth)

4. Middleware
   - Create auth middleware for protected routes
   - Add error handling for invalid tokens

5. Testing
   - Write unit tests for auth service
   - Write integration tests for endpoints

Estimated files to create/modify: 8 files

Do you approve this plan? (yes/no/modify)"

You: "yes"

Claude: "Executing plan..."
[1/5] Creating users table migration... ✓
[2/5] Implementing authentication service... ✓
[3/5] Adding API endpoints... ✓
[4/5] Creating auth middleware... ✓
[5/5] Writing tests... ✓

"Authentication implemented successfully!

Summary:
- Added 8 files
- Modified 2 files
```

```
- All tests passing (12/12)

Try it:
POST /register with {"email":"user@example.com", "password":"secret"}
POST /login to get JWT token"
```

Example 2: Refactoring

```
You: "Refactor this code to use dependency injection"

Claude (Plan Mode):
"I'll refactor to use dependency injection. Here's my plan:

📋 PLAN:

1. Create Dependency Container
   - Add dependency injection framework (inject)
   - Create container.py for registrations

2. Refactor Services
   - UserService: inject database
   - EmailService: inject config
   - PaymentService: inject UserService & EmailService

3. Update API Routes
   - Modify routes to get services from container
   - Remove direct instantiation

4. Update Tests
   - Create mock services
   - Update tests to inject mocks

Files to modify: 6 files
New files: 1 file (container.py)

This follows the Dependency Inversion Principle (SOLID).

Approve? (yes/no/modify)"
```

Example 3: Bug Fix

```
You: "Fix the NullPointerException in UserService"

Claude (Plan Mode):
"I'll investigate and fix the NullPointerException. Here's my plan:

📋 PLAN:

1. Investigation
```

```
   - Read error logs
   - Identify exact line causing NPE
   - Trace where null value originates

2. Root Cause Analysis
   - Check if null is expected behavior
   - Determine if defensive check or upstream fix needed

3. Fix Implementation
   - Add null validation where appropriate
   - Update upstream to prevent null if needed
   - Add meaningful error messages

4. Testing
   - Add unit test reproducing the bug
   - Verify fix resolves issue
   - Add tests for edge cases

5. Prevention
   - Add similar checks to related code
   - Update documentation if needed

Estimated time: 15 minutes

Approve? (yes/no/modify)"
```

Best For

- Learning how AI approaches problems
- Complex tasks requiring verification
- Critical production changes
- When you want control at each step
- Understanding the strategy before execution

---

3. Auto Mode (Autonomous Execution)

What It Is

- Immediate autonomous execution
- No plan shown upfront
- Executes task end-to-end
- Reports results when complete

When to Use

- Well-defined tasks
- Trust the approach
- Want speed over visibility
- Non-critical changes

- Repetitive tasks

Capabilities

☑ Immediate execution ☑ Edit files autonomously ☑ Run commands autonomously ☑ Debug and fix issues ☑ Iterate until complete ❌ No plan preview ❌ Less visibility during execution

Workflow

```
1. You describe task
2. Claude immediately starts executing
3. Claude edits files, runs commands, debugs
4. Claude reports when complete
```

Examples

Example 1: Quick Feature

```
You: "Add a /health endpoint that returns server status"

Claude (Auto Mode):
[Immediately starts working]

"Adding health endpoint..."

✓ Created routes/health.py
✓ Added health check logic
✓ Registered route in main.py
✓ Added test_health.py

"Health endpoint added successfully!

GET /health returns:
{
  "status": "healthy",
  "timestamp": "2026-08-05T10:00:00Z",
  "database": "connected",
  "redis": "connected"
}

Test it: curl http://localhost:8000/health"
```

Example 2: Bug Fix

```
You: "Fix the bug where users can't update their profile"

Claude (Auto Mode):
[Starts investigating]
```

```
"Investigating profile update bug..."
- Checking error logs... found 500 error
- Reading UserController.update_profile()... found issue
- Missing validation causes database constraint violation

"Fixing bug..."
✓ Added input validation
✓ Fixed database query
✓ Added error handling
✓ Updated tests

"Bug fixed! Users can now update profiles.
Tested with user ID 123 - update successful."
```

Example 3: Add Tests

```
You: "Add unit tests for the payment service"

Claude (Auto Mode):
"Adding unit tests for PaymentService..."

✓ Created tests/test_payment_service.py
✓ Added 12 test cases:
  - test_successful_payment
  - test_payment_declined
  - test_insufficient_funds
  - test_invalid_card
  - test_refund_success
  - test_refund_failed
  - test_payment_timeout
  - test_duplicate_payment
  - test_amount_validation
  - test_currency_validation
  - test_metadata_storage
  - test_webhook_handling

"All tests passing (12/12)
Coverage: 94% for payment_service.py

Run tests: pytest tests/test_payment_service.py"
```

Best For

- Quick tasks
- Well-understood problems
- When you trust the approach
- Speed is priority
- Non-critical code

- Repetitive work

---

4. Cascade Mode (Multi-File Autonomous)

What It Is

- Most powerful autonomous mode
- Edits multiple files simultaneously
- Understands entire codebase context
- Implements complete features
- Runs tests and debugs automatically

When to Use

- Large features spanning many files
- Complex refactorings
- Architecture changes
- Adding new modules
- When you can step away for a while

Capabilities

☑ Multi-file editing ☑ Autonomous planning and execution ☑ Reads entire codebase for context ☑ Runs tests automatically ☑ Fixes failing tests ☑ Iterates until task complete ☑ Handles dependencies ☑ Updates documentation

Workflow

```
1. You describe high-level feature/task
2. Claude analyzes entire codebase
3. Claude edits multiple files simultaneously
4. Claude runs tests
5. Claude fixes issues found
6. Claude iterates until complete
7. Reports comprehensive results
```

Examples

Example 1: Complete Feature

```
You: "Build a REST API for blog posts with full CRUD operations"

Claude (Cascade Mode):
[Analyzing codebase...]
"Found existing patterns in:
- models/ for database models
- routes/ for API endpoints
```

```
- services/ for business logic
- tests/ for testing

Building blog post API..."

[Editing multiple files simultaneously]

✓ Created models/blog_post.py
  - BlogPost model with title, content, author, timestamps
  - Relationships with User model

✓ Created migrations/001_add_blog_posts.py
  - Creates blog_posts table
  - Adds foreign key to users

✓ Created services/blog_service.py
  - create_post(title, content, author_id)
  - get_post(post_id)
  - update_post(post_id, data)
  - delete_post(post_id)
  - list_posts(filters)

✓ Created routes/blog.py
  - POST /posts - create post
  - GET /posts/{id} - get post
  - PUT /posts/{id} - update post
  - DELETE /posts/{id} - delete post
  - GET /posts - list posts (with pagination)

✓ Updated main.py
  - Registered blog routes

✓ Created tests/test_blog_service.py
  - 8 test cases for service layer

✓ Created tests/test_blog_routes.py
  - 10 test cases for API endpoints

✓ Updated docs/API.md
  - Added blog post endpoints documentation

[Running tests...]
✓ All tests passing (18/18)

[Running migration...]
✓ Database updated

"Blog post API complete!

Summary:
- 7 files created
- 2 files modified
- 18 tests passing
- Database migrated
```

```
Try it:
POST /posts {"title":"Hello","content":"World","author_id":1}
GET /posts
GET /posts/1
PUT /posts/1 {"title":"Updated"}
DELETE /posts/1"
```

Example 2: Microservices Architecture

```
You: "Split this monolith into user service and order service microservices"

Claude (Cascade Mode):
[Analyzing monolith structure...]
"Found:
- 45 files in monolith
- User-related: 12 files
- Order-related: 18 files
- Shared: 15 files

Creating microservices architecture..."

[Creating new directory structure]

user-service/
✓ models/user.py (extracted)
✓ routes/user_routes.py (extracted)
✓ services/user_service.py (extracted)
✓ main.py (new)
✓ Dockerfile (new)
✓ requirements.txt (new)
✓ tests/ (extracted & modified)

order-service/
✓ models/order.py (extracted)
✓ routes/order_routes.py (extracted)
✓ services/order_service.py (extracted)
✓ clients/user_client.py (new - calls user-service)
✓ main.py (new)
✓ Dockerfile (new)
✓ requirements.txt (new)
✓ tests/ (extracted & modified)

shared/
✓ auth/jwt.py (shared)
✓ database/connection.py (modified for separate DBs)
✓ models/base.py (shared)

infrastructure/
✓ docker-compose.yml (new - orchestrates services)
✓ nginx.conf (new - API gateway)
✓ .env.example (new)
```

```
[Running tests...]
✓ user-service tests: 24/24 passing
✓ order-service tests: 32/32 passing

[Building Docker containers...]
✓ user-service built
✓ order-service built

"Microservices architecture complete!

Summary:
- Created 2 independent services
- Extracted 30 files
- Created 15 new files
- Modified 8 shared files
- All tests passing (56/56)
- Docker-ready

Start services:
docker-compose up

Endpoints:
- User Service: http://localhost:8001
- Order Service: http://localhost:8002
- API Gateway: http://localhost:8080

Migration guide created in MIGRATION.md"
```

Example 3: Performance Optimization

```
You: "Optimize this app for high performance - currently handles 100 req/s, need
1000 req/s"

Claude (Cascade Mode):
[Analyzing performance bottlenecks...]

"Found issues:
1. N+1 database queries (5 locations)
2. No caching layer
3. Synchronous external API calls
4. Missing database indexes
5. Inefficient JSON serialization

Implementing optimizations..."

[Multi-file optimization]

✓ Added Redis caching layer
  - cache/redis_client.py
  - cache/decorators.py
  - Caches user data (TTL: 5min)
```

```
  - Caches product catalog (TTL: 1hr)

✓ Fixed N+1 queries
  - Modified 5 database queries
  - Added eager loading
  - Reduced queries from 150 to 8 per request

✓ Made external calls async
  - Converted payment API to async
  - Converted email service to async
  - Parallelized independent calls

✓ Added database indexes
  - Index on users.email
  - Index on orders.user_id
  - Index on orders.created_at
  - Composite index on (user_id, status)

✓ Optimized serialization
  - Replaced json.dumps with orjson
  - Added Pydantic for validation
  - Reduced serialization time by 70%

✓ Added connection pooling
  - Database pool: 20 connections
  - Redis pool: 10 connections
  - HTTP client pool: 50 connections

✓ Updated tests for async
  - Modified 24 test cases
  - All tests passing

[Running benchmarks...]
Before: 120 req/s, p95: 850ms
After:  1250 req/s, p95: 95ms

"Performance optimization complete!

Improvements:
- Throughput: 10x increase (120 → 1250 req/s)
- Latency: 9x decrease (850ms → 95ms)
- Database queries: -95% (150 → 8 per request)

Summary:
- 18 files modified
- 6 new files
- 4 database indexes added
- All tests passing (56/56)

Load test results in BENCHMARKS.md"
```

Best For

- Complete features
- Large refactorings
- Multiple related changes
- When you can step away
- Complex implementations
- Greenfield projects

Mode Comparison

Feature	Vibe	Plan	Auto	Cascade
File Edits	❌	☑	☑	☑
Command Execution	❌	☑	☑	☑
Shows Plan	N/A	☑	❌	❌
Multi-File	❌	Limited	Limited	☑
Autonomous	❌	Semi	☑	☑
Requires Approval	N/A	☑	❌	❌
Speed	Fast	Medium	Fast	Medium
Visibility	High	High	Medium	Low
Best For	Questions	Planning	Quick tasks	Features

Mode Selection Decision Tree

```
Do you need code changes?
├─ No → VIBE MODE (questions, explanations)
└─ Yes
   │
   ├─ Is it a complex feature?
   │  ├─ No → Single file change?
   │  │  ├─ Yes → AUTO MODE (quick fix)
   │  │  └─ No → PLAN MODE (multi-step)
   │  └─ Yes → Multiple files?
   │     ├─ Yes → CASCADE MODE (feature)
   │     └─ No → PLAN MODE (verify approach)
   │
   ├─ Do you want to see the plan first?
   │  ├─ Yes → PLAN MODE
   │  └─ No → AUTO or CASCADE
   │
   └─ Is this critical production code?
      ├─ Yes → PLAN MODE (review first)
      └─ No → AUTO or CASCADE
```

Real-World Usage Patterns

Morning: Bug Triage

1. **Vibe**: "Explain what this error means"
2. **Plan**: "Fix the authentication bug" (review approach)
3. **Auto**: "Add logging to track the issue"

Midday: Feature Development

1. **Vibe**: "Should I use webhooks or polling?"
2. **Cascade**: "Implement webhook handler for Stripe payments"

Afternoon: Code Review

1. **Vibe**: "Review this pull request for issues"
2. **Auto**: "Fix the linting errors"
3. **Plan**: "Refactor for better testability"

End of Day: Cleanup

1. **Auto**: "Add type hints to user_service.py"
2. **Auto**: "Update documentation"
3. **Cascade**: "Add tests for all new features today"

Mode Switching

You can switch modes mid-conversation:

```
You (Vibe): "How should I implement caching?"

Claude: "I recommend Redis with these patterns..."

You: "Switch to Cascade mode and implement it"

Claude (Cascade): "Switching to Cascade mode.
Implementing Redis caching..."
```

Advanced Features

Cascade Flows (Saved Workflows)

Create reusable workflows:

```yaml
name: "Add REST Endpoint"
mode: cascade
```

```
steps:
  - Create model
  - Create service
  - Create route
  - Add validation
  - Write tests
  - Update docs
```

Then trigger: "Run 'Add REST Endpoint' flow for Products"

Context Control

Control what Claude sees:

```
@file:auth.py - reference specific file
@folder:services - reference folder
@codebase - entire codebase
@docs:stripe - external documentation
```

Checkpoints

In Cascade mode, create checkpoints:

```
You: "Implement user service in Cascade mode,
     but create checkpoint after each major component"

Claude:
✓ Checkpoint 1: Models created
✓ Checkpoint 2: Services created
✓ Checkpoint 3: Routes created
← Checkpoint 4: Tests created (you are here)
  Checkpoint 5: Documentation

Rollback available to any checkpoint.
```

Best Practices

Vibe Mode

☑ Use for learning and planning ☑ Ask follow-up questions ☑ Request examples and explanations ✗ Don't expect code changes

Plan Mode

☑ Review plan before approving ☑ Request modifications if needed ☑ Use for critical changes ✗ Don't skip plan review for important code

Auto Mode

☑ Use for quick, well-defined tasks ☑ Trust for non-critical changes ☑ Good for repetitive work ❌ Don't use for complex features

Cascade Mode

☑ Use for complete features ☑ Can step away during execution ☑ Let it iterate and fix issues ❌ Don't interrupt mid-execution ❌ Don't use for critical production without review

---

Troubleshooting

Cascade Gets Stuck

```
You: "Status check - where are you?"

Claude: "Currently on step 3/5: Writing tests
Having issue with mock setup, trying alternative approach..."

You: "Take a different approach - skip mocks, use real DB in tests"

Claude: "Adjusting approach, continuing..."
```

Want to Change Plan Mid-Execution

```
You: "Stop - change plan to use PostgreSQL instead of MySQL"

Claude: "Stopping current execution.
Updated plan:
- Reverting MySQL changes
- Switching to PostgreSQL
- Updating dependencies

Continue? (yes/no)"
```

Review Cascade Changes Before Committing

```
You: "Show me all files changed"

Claude: "Changes made:
Modified: 8 files
Created: 12 files
Deleted: 2 files

[Shows diff for each file]
```

```
Approve and commit? (yes/no/show-specific)"
```

Interview Answer Template

"What mode would you use in Claude Code for X?"

Framework:

1. Identify if it's a question or implementation
2. Assess complexity and file count
3. Consider need for oversight
4. Choose mode and justify

Example:

"For implementing a payment integration feature, I'd use **Cascade mode** because:

- It spans multiple files (models, service, routes, tests)
- It's well-defined (Stripe API integration)
- I can let it work autonomously
- It will handle dependencies and test failures

I wouldn't use:

- Vibe: Need actual implementation
- Plan: Too many files, approval overhead
- Auto: Too complex for single-shot execution

After Cascade completes, I'd review the changes and test thoroughly before deploying."

Summary

Mode	When to Use	Key Benefit
Vibe	Questions, learning, planning	Understanding
Plan	Multi-step with oversight	Control
Auto	Quick defined tasks	Speed
Cascade	Complete features	Power

Start with Vibe to understand → **Use Plan** for important changes → **Use Auto** for quick tasks → **Level up to Cascade** for complex features

Master one mode at a time, gradually adopt more powerful modes as you build trust.